cd510037-a2ff-4bc2-982e-578bcd5e0b50

<current>

<request>

You are working on an issue in the 'Insight-Services-APAC/ingenious' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ingenious/ingenious (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-20T08:22:11.861Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>Performance profiling and optimization baseline</issue_title>
<issue_description>## Problem
No baseline performance metrics exist. We don't know:
- How long typical requests take
- Where bottlenecks are
- What the resource usage profile looks like
- If optimizations are needed

Goals
1. Establish performance baselines
2. Identify bottlenecks
3. Create monitoring strategy
4. Optimize critical paths if justified

```
## Suspected Bottlenecks

### 1. Dynamic Import Overhead
(\`ingenious/utils/imports.py\`)
**Hypothesis**: \`import_class_with_fallback()\` runs on every
request

**Potential Fix**: Cache imported classes
\`\`\`python
_import_cache: Dict[Tuple[str, str], type] = {}

def import_class_with_fallback(module, class_name):
    cache_key = (module, class_name)
    if cache_key in _import_cache:
        return _import_cache[cache_key]
    # ... import logic ...
    _import_cache[cache_key] = cls
    return cls
\`\`\`

### 2. Message History Loading
(\`ingenious/services/chat_services/multi_agent/
service.py:95-106\`)
**Hypothesis**: Loading last 10 messages on every request is
expensive

**Potential Fix**:
- Add database index on (thread_id, timestamp)
- Use LIMIT in SQL, not Python slicing
- Cache recent messages

### 3. No Request Timeouts
**Hypothesis**: Long-running LLM calls can hang

**Potential Fix**: Add timeout middleware

### 4. Database Query Optimization
**Investigate**: Are there indexes? Using SELECT *?

## Deliverables

### 1. Baseline Document (\`docs/performance/baseline.md\`)
\`\`\`markdown
# Performance Baseline (v0.2.7)

## Benchmark Results
- Simple chat: p50=XXms, p95=XXms
```

```
- With history: p50=XXms, p95=XXms
- Streaming: TTFBXms

## Bottlenecks
1. Dynamic imports: XXms
2. Message loading: XXms

## Optimizations
1. [Name] - [Impact]
\`\`\`

### 2. Profiling Scripts (\`scripts/profiling/\`)
- \`profile_request.py\`
- \`profile_streaming.py\`
- \`load_test.py\`
- \`memory_profile.py\`

## Acceptance Criteria
- [ ] Baseline document created
- [ ] Profiling scripts created
- [ ] 3+ bottlenecks identified
- [ ] 1+ optimization implemented (if justified)
- [ ] Memory profiling (no leaks found)
- [ ] Database performance analyzed
- [ ] Monitoring strategy proposed

## Tools
\`\`\`bash
uv add --dev py-spy memory-profiler line-profiler
\`\`\`

## Success Metrics
- Request latency reduced &gt;20% (if optimizations made)
- No memory leaks
- Clear performance characteristics documented</
issue_description>

## Comments on the Issue (you are @copilot in this section)

<comments>
</comments>

</problem_statement>
```

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.